



T.C.

HALIÇ UNIVERSITY

FACULTY OF BUSINESS ADMINISTRATION

MANAGEMENT INFORMATION SYSTEMS

**PERFORMANCE IMPROVEMENT IN SPRING BOOT
APPLICATIONS USING REDIS CACHING**

GRADUATION PROJECT

Emirhan KAYA

21051350137

Project Advisor: Prof. Dr. Nuray TEZCAN

2026

PLAGIARISM REPORT

PERFORMANCE IMPROVEMENT IN SPRING BOOT APPLICATIONS USING REDIS CACHING.docx

ORIGINALITY REPORT

1 %	1 %	1 %	1 %
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	ijeecs.iaescore.com Internet Source	1 %
2	Submitted to Griffth University Student Paper	1 %

Exclude quotes	On	Exclude matches	< 1%
Exclude bibliography	On		

PREFACE

This study was completed as part of the graduation project requirement at Haliç University, Management Information Systems Department. I would like to express my sincere gratitude to my advisor Prof. Dr. Nuray Tezcan for her guidance and support throughout this process.

Emirhan Kaya

TABLE OF CONTENTS

	<u>Page No</u>
1. INTRODUCTION	1
2. LITERATURE RESEARCH	2
2.1. Web Application Performance	2
2.2. Caching Concept and Strategies	2
2.3. Redis Architecture and Features	4
2.4. Spring Boot and Caching Integration	4
2.5. Previous Performance Studies	5
2.6. Performance Testing Methodology	6
3.METHODOLOGY	7
3.1. System Architecture	7
3.2. Test Environment	10
3.3. Dataset	10
3.4. Test Scenarios	11
3.5. Performance Metrics	14
4. FINDINGS	15
4.1. Test Results	15
4.2. Analysis	15
5. CONCLUSION AND RECOMMENDATIONS	17
5.1. Summary of Findings	17
5.3. Practical Implications	18
5.4. Limitations	18

ABBREVIATIONS

API : Application Programming Interface

AOP : Aspect-Oriented Programming

CRUD : Create, Read, Update, Delete

DBMS : Database Management System

HTTP : Hypertext Transfer Protocol

JPA : Java Persistence API

RAM : Random Access Memory

RDBMS : Relational Database Management System

REST : Representational State Transfer

SQL : Structured Query Language

TTL : Time to Live

CSV : Comma-Separated Values

HIT : Cache Hit

JSON : JavaScript Object Notation

MISS : Cache Miss

URL : Uniform Resource Locator

LIST OF TABLES

	<u>Page</u>
Table 1. Query Execution Times With and Without Redis Caching	5
Table 2. Database Structure	11
Table 3. Test Scenarios	11
Table 4. Test Results by Scenario	15
Table 5. Detailed Performance Metrics by Scenario	16

LIST OF FIGURES

	<u>Page No</u>
Figure 1. Application Layer Architecture	7
Figure 2. Web Interface — Scenario Test Panel	8
Figure 3. Web Interface — API Response and Cache Status	9
Figure 4. Web Interface — Test Results Dashboard	10
Figure 5. Request Flow Diagram for All Test Scenarios	13

ABSTRACT IN ENGLISH

PERFORMANCE IMPROVEMENT IN SPRING BOOT APPLICATIONS USING REDIS CACHING

In web-based applications, user requests typically require retrieving data from a database. In short, a database is a storage system where the application permanently stores all its data. Every time a query goes to and returns from this system, it takes some time—this is called latency. As the number of users, the complexity of the requested data, and the volume of data increase, the cost of these round trips rises, and response times lengthen. This is one of the primary factors determining the application's perceived performance in the eyes of users. Various solutions have been developed to reduce response times and use system resources more efficiently. Caching is one of the most widely used of these solutions.

In this study, an application was developed using Spring Boot to measure the impact of caching on application performance. The process can be summarized as follows: First, the controller layer handled incoming requests then the service layer checked the Redis cache if it is activated or not. That's because caching can be enabled or disabled.

Three scenarios were tested and each scenario has its own settings. These scenarios can be listed as: Baseline, Cold Cache and Warm Cache. In the Baseline scenario, caching was completely disabled. So, each user request was sent directly to the database by service layer. In the Cold Cache scenario, caching was enabled but Redis was started empty. So that's means the first requests were directed to the database like the baseline scenario because there was no data in the cache, and the results were then written to Redis to read from there in the later requests. In the Warm Cache scenario, the cache was full, and all incoming requests were answered directly from Redis. Each scenario was run with 100 concurrent users and 10,000 requests.

The tests were made in a local environment using PostgreSQL 16 and Redis 7 that installed via docker. The application was developed using Spring Boot 3.x and Java 21. All tests were automatically recorded by Apache JMeter in CSV format, and all the calculations and tables use these stats. These are the results of the tests: In the Baseline scenario, the average response time was at 1.21 ms, and the maximum response time was 51 ms. In the Cold Cache scenario, as expected the average response time increased to 1.42 ms that is because system needed to

write cache to redis before returning a answer since the caching was enabled, also the maximum response time decreased to 19 ms. Finally, In the Warm Cache scenario, since data is served from RAM, PostgreSQL was never used because caching was enabled and redis was full, the average response time dropped to 0.71 ms, and the maximum time was 14 ms. This shows us to a 41% improvement compared to the Baseline scenario.

The findings finds that Redis caching can significantly reduce response times. However, this benefit depends on how much data is pre-loaded in the cache before we sending requests. If Redis is configured for the application's data and most used data, performance increases much more.

Keywords : *Redis, Caching, Spring Boot, Database, Performance.*

ABSTRACT IN TURKISH

PERFORMANCE IMPROVEMENT IN SPRING BOOT APPLICATIONS USING REDIS CACHING

Web tabanlı uygulamalarda kullanıcı istekleri çoğunlukla bir veritabanından veri çekmeyi gerektirir. Veritabanı kısaca, uygulamanın tüm verilerini kalıcı olarak sakladığı bir depolama sistemidir. Her sorgu bu sisteme gidip geri döndüğünde bir süre harcanır — buna gecikme (latency) denir. Kullanıcı sayısı, talep edilen veri, bu verinin ne kadar detaylı olduğu ve istenen verinin miktarı arttıkça bu gidiş-dönüşlerin yani istek ve dönen yanıtların maliyeti de artar ve yanıt süreleri uzar. Bu da uygulamanın performansı hakkında kullanıcıların zihninde fikir oluşturan önemli etkenlerden biridir. Yanıt sürelerini kısaltmak ve sistem kaynaklarını daha verimli kullanmak amacıyla çeşitli çözümler geliştirilmiştir. Önbellekleme (caching) bu çözümlerin en yaygın kullanılanlarından biridir.

Bu çalışmada, önbelleklemenin uygulama performansı üzerindeki etkisini ölçmek amacıyla Spring Boot kullanılarak bir uygulama geliştirilmiştir. Geliştirilen uygulamada controller katmanı gelen istekleri karşılamış, servis katmanı önce Redis önbelleğini kontrol etmiş, veri bulunamadığında PostgreSQL'e yönlendirmiştir. Önbellekleme davranışı çalışma zamanında etkinleştirip devre dışı bırakılabilecek şekilde tasarlanmıştır.

Apache JMeter ile üç senaryo test edilmiştir: önbelleklemenin kapalı olduğu Baseline, önbelleğin boş başladığı Cold Cache ve önbelleğin dolu olduğu Warm Cache. Baseline senaryosunda önbellekleme tamamen devre dışı bırakılmıştır; her kullanıcı isteği doğrudan PostgreSQL veritabanına iletilmiş ve veriler diskten okunarak yanıt oluşturulmuştur. Cold Cache senaryosunda önbellekleme etkinleştirilmiş ancak Redis boş olarak başlatılmıştır; ilk gelen istekler önbellekte veri bulamadığından veritabanına yönlendirilmiş, ardından elde edilen sonuç Redis'e yazılmıştır. Warm Cache senaryosunda ise önbellek dolu halde bırakılmış ve gelen tüm istekler veritabanına hiç ulaşmadan doğrudan Redis'ten yanıtlanmıştır. Her senaryo 100 eş zamanlı kullanıcı ile 10.000 istek üzerinden çalıştırılmıştır.

Testler yerel bir ortamda gerçekleştirilmiştir ve PostgreSQL 16 ve Redis 7 Docker kullanılarak kurulmuştur. Uygulama Spring Boot 3.x ve Java 21 kullanılarak geliştirilmiştir. Tüm ölçümler Apache JMeter tarafından otomatik olarak CSV formatında kaydedilmiş ve yanıt süreleri milisaniye cinsinden raporlanmıştır. Baseline senaryosunda ortalama yanıt süresi 1.21 ms,

maksimum yanıt süresi ise 51 ms olarak ölçülmüştür. Cold Cache senaryosunda her önbellek isabetsizliği veritabanı sorgusu ve ardından Redis'e yazma işlemi gerektirdiğinden ortalama yanıt süresi 1.42 ms'ye yükselmiş, ancak maksimum süre 19 ms'ye gerilemiştir. Warm Cache senaryosunda veriler doğrudan RAM'den sunulduğu için disk erişimi tamamen ortadan kalkmış, ortalama yanıt süresi 0.71 ms'ye düşmüş ve maksimum süre 14 ms olarak gerçekleşmiştir. Bu sonuç Baseline'a kıyasla yüzde 41 iyileşmeye karşılık gelmektedir.

Elde edilen bulgular, Redis önbelleklemenin yanıt sürelerini önemli ölçüde kısaltabildiğini göstermektedir. Ancak bu kazanım, önbellekte ne kadar verinin hazır olduğuna bağlıdır. Redis, uygulamanın veri yapısına ve en çok erişilen verilere göre doğru ayarlandığında performans artışı çok daha net görülecektir.

Keywords : *Redis, Caching, Spring Boot, Database, Performance*

1. INTRODUCTION

Web applications play a key role in modern business, and their performance has a direct impact on users and companies that use them. As the amount of data grows, normally the response times increase and it becomes harder to keep response times fast. One of the significant reasons for this problem is sending the same database requests again and again. It reduces the application performance.

There are some technologies to solve this problem and Caching is one of them. Since we have plenty of options for caching systems I decided to use Redis cache System. It's a very popular tool for caching systems generally used Java programming language.

Java is one of the most widely used programming languages for backend development and it has some frameworks such as Spring Boot, built on top of the Java ecosystem. Spring Boot has become the leading framework for developing REST APIs and web services. It basically packages all the things you need to develop a web application. For example, when creating a web interface developed in this project in a local environment, There was no need to go through setting up a server because Spring Boot handles that for us. Additionally, when filling database by Java Faker library Spring Boot's dependency system was used.

The purpose of this study is to test performance impact of Redis Cache by measuring response times with and without caching enabled, in order to identify the improvement that Redis caching provides.

2. LITERATURE RESEARCH

2.1. Web Application Performance

So basically the performance of web-based systems is the most critical part of it. From user's perspective, performance means how quickly a web page works and can be used to get negative or positive idea about your business. Although many components and systems work together in the background, what is important for the user is how fast the system is reacting his/her actions. Poor performance can lead to serious reputational and financial damage for companies operating online (Privalov & Stupina, 2024).

The performance of web applications depends on several factors such as application architecture, the database management system used, server hardware, and code quality (Privalov & Stupina, 2024). Among these, the database plays a particularly important role since it handles storing and getting the main data.

Data access is often the main source of response delays in web applications, as it involves disk operations, filtering, and sorting large amounts of data (Privalov & Stupina, 2024). Repeated and unnecessary database requests put a heavy load on the system, and studies have shown that most performance bottlenecks in Java-based applications originate in the data access layer (Bonteanu & Tudose, 2024). As user and data amount increase, the weaknesses of traditional database systems become more visible, making performance optimization at the data access layer very critical for web applications.

2.2. Caching Concept and Strategies

Caching is like putting our summer clothes on the front shelves when summer comes. It is a method of temporarily storing the most accessed data in a fast storage layer, such as RAM, so that it can be accessed more quickly without repeating the same costly operations. Mertz and Nunes (2020) define application-level caching as the addition of caching logic into application code in order to temporarily store processed content in memory, allowing it to be reused and reducing response times. When the application returns the data directly from the cache, applications avoid unnecessary load on the database in this study it is PostgreSQL, which

significantly improves overall system efficiency and addresses the performance bottlenecks discussed above.

Several caching strategies exist such as the cache-aside strategy, also known as lazy loading, only stores data in the cache when it is actually requested. On a cache miss, the application retrieves data directly from the primary data source and then writes it to the cache for future use. Although this approach may result in slightly slower response times on the first request, it is the most widely adopted strategy due to its simplicity and efficiency (Kanthed, 2023).

In the write-through strategy, data is written to both the cache and the database, ensuring that the cache always contains newest information. The write-behind strategy writes data to the cache first and asynchronously synchronizes it with the database. While write-behind improves write performance, it carries a risk of data loss if the cache fails before synchronization occurs (Kanthed, 2023). Among these strategies, cache-aside is better for read-heavy web applications such as developed in this study, and research has confirmed that Redis handles cache-aside reads with sub-millisecond latency (Deshpande, 2025).

Disk-based databases store data on physical storage devices such as hard drives. This means that every time the application needs data, it has to go to hard drive and ask it, which is a slow process. When many users send requests at the same time, this becomes a problem and can cause delays in the application. Caching systems such as Redis keep all data in RAM, which is much faster to access than a disk-based hard drives. This helps the application to respond to requests much more quickly (Ayub & Ali, 2018). Studies have shown that switching from disk-based to in-memory storage significantly reduces response times, especially under heavy user load (Ayub & Ali, 2018). It is a significant fact that in-memory storage loses its data when the server restarts. Although this is not a major concern for caching purposes, since the original data remains in the primary database. These are the reasons of in-memory solutions are generally preferred as a caching layer in web applications. And among these solutions, Redis has become one of the most widely used tools because of its speed, flexibility, and ability to work with different data structures and types. (Kanthed, 2023).

2.3. Redis Architecture and Features

Redis is a data storage tool that keeps all data in the computer's memory (RAM) instead of saving it to disk. Because of the nature and features of RAM it can respond to requests very quickly. It handles many requests at the same time using a single processing loop, which keeps things simple and fast (Kanthed, 2023). This makes Redis one of the most popular caching solutions used in web applications today.

Also, Redis is not just a simple key-value store such as dictionaries in programming languages. Redis can work with many different types of data such as lists, sets, and sorted sets. This gives developers more flexibility when building caching solutions (Kanthed, 2023). Redis also has a feature that lets save our data to disk so that the data is not lost if the server shuts down. It can do this by saving snapshots of the data at regular intervals, or by keeping a log of every change that is made to the data (Kanthed, 2023).

In addition, there are some other important features. When the cache gets full, Redis can automatically delete data that has not been used recently (Kanthed, 2023). Redis is often compared to Memcached which is another caching tool that is simple and faster for basic tasks. Redis has more features like data persistence and replication. This makes Redis a better choice when the application needs both speed and reliability (Kanthed, 2023).

2.4. Spring Boot and Caching Integration

Spring boot is a Java Framework that allows users to build web applications. It speeds up the setup work by using its automatic configuration. Because of this, it has become one of the most widely used frameworks for backend development in enterprise applications (Privalov & Stupina, 2024).

Also, Spring Boot has caching system so developers can integrate caching to their web application without complex setup process. This can be done by adding simple annotations directly to methods. The most common annotation is `@Cacheable`, which saves the result of a method to the cache the first time it is called. So, when the same method is called again the result is returned directly from the cache. This helps the application to answer quicker. There are some other important annotations such as `@CacheEvict`, which removes data from the cache when it is no longer needed or when the data in the database changes (Privalov & Stupina, 2024).

Integrating Redis as a caching system into Spring Boot is quite easy. The steps to follow can be summarized as follows: Add the necessary dependencies and configure the settings; after that, Spring Boot will automatically start using Redis. Research has also shown that introducing in-memory caching solutions such as Redis into microservice-based applications helps avoid redundant computations and reduces memory spikes under heavy load (Henao Villa et al., 2025). In a study that built a Spring Boot application with Redis caching, it was shown that this integration significantly reduced the number of database queries and improved overall response times (Privalov & Stupina, 2024).

2.5. Previous Performance Studies

Studies have been conducted to measure the performance of the Redis caching. As an example Privalov and Stupina (2024) conducted tests using various scenarios on a table containing over 120,000 records. The results of these scenarios are shown in the table 1 below.

Table 1. Query Execution Times With and Without Redis Caching

Caching Strategy	Without Caching (ms)	With Caching (ms)	Speedup
Unsigned read/write – Key	197.5	41.2	~4.8x
Read/write – Key	397.4	71.0	~5.6x
Unsigned read/write – Serialized	197.5	30.0	~6.6x
Read/write – Serialized	397.4	60.0	~6.6x

Source: Adapted from Privalov & Stupina (2024)

The TTL is used in the unsigned read/write strategy. This means that after a certain period of time, the data cannot be modified. In other words, it has a limited life but the read/write strategy allows for updates. Upon reviewing the results, we observed significant improvements in performance and response times across the scenarios. We can easily see the improvement in the application's response time by looking at the "Speedup" column. In summary, it appears that the impact of the redesign can generally be viewed as positive. As shown in the table 1 caching improves the performance of the application by 4.8x to 6.6x.

2.6. Performance Testing Methodology

In various studies, and particularly in the study conducted by Privalov & Stupina (2024), numerous tests were performed using JMeter. JMeter is an application that allows developers to send a large number of requests to a server. This allows us to test how our applications behave under heavy loads and requests and to improve the application. According to Yenugula et al. (2019), JMeter can be described as a software application that analyzes how a server operates under load and assesses its performance based on specific conditions. The most important metric for us here is response time. Response time can be summarized as the duration from the moment we perform an action. For example, clicking a button, until the backend processes the code blocks, databases, and other systems and returns a response to us. In other word, the time period between sending a request and receiving a reply.

3.METHODOLOGY

3.1. System Architecture

The web application used in this study was developed using Spring Boot 3.x and Java 21. As shown in Figure 1, to summarize the request-response process: In the controller layer, incoming requests are routed to service classes based on the scenario information provided in the request, and if necessary, they are directed to the database or the cache system to return a response to the user.

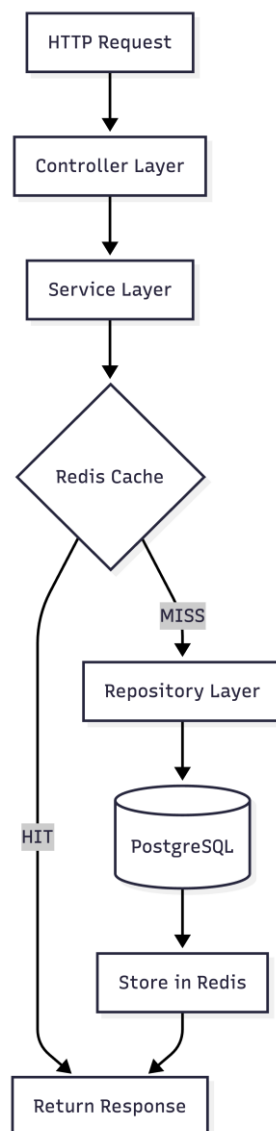


Figure 1. Application Layer Architecture

All tests were carried out on the GET /api/products/{id} endpoint under different scenarios. Since the focus of this study is the effect of Redis caching, dedicated endpoints were implemented to enable and disable the cache at runtime. In addition, endpoints were set up for use in scenario preparation processes and to speed up the setup of these processes

An “X-Cache-Status” header was added to each response. If the data came from Redis, this header was set to “HIT.” If it came directly from the database (i.e., PostgreSQL), the header was sent as a “MISS.”

The tests used in this study were conducted using Jmeter, and the data from these tests was used in all statistical analyses. However, a simple web interface was also designed to make the results visible to users. Through this interface, test scenarios can be run manually and the results can be viewed.

Figure 2 shows a screenshot of the scenario test panel. Here, scenario can be launched , and the results are displayed as a bar chart directly below. The image below shows the “warm cache” scenario in the bar chart.

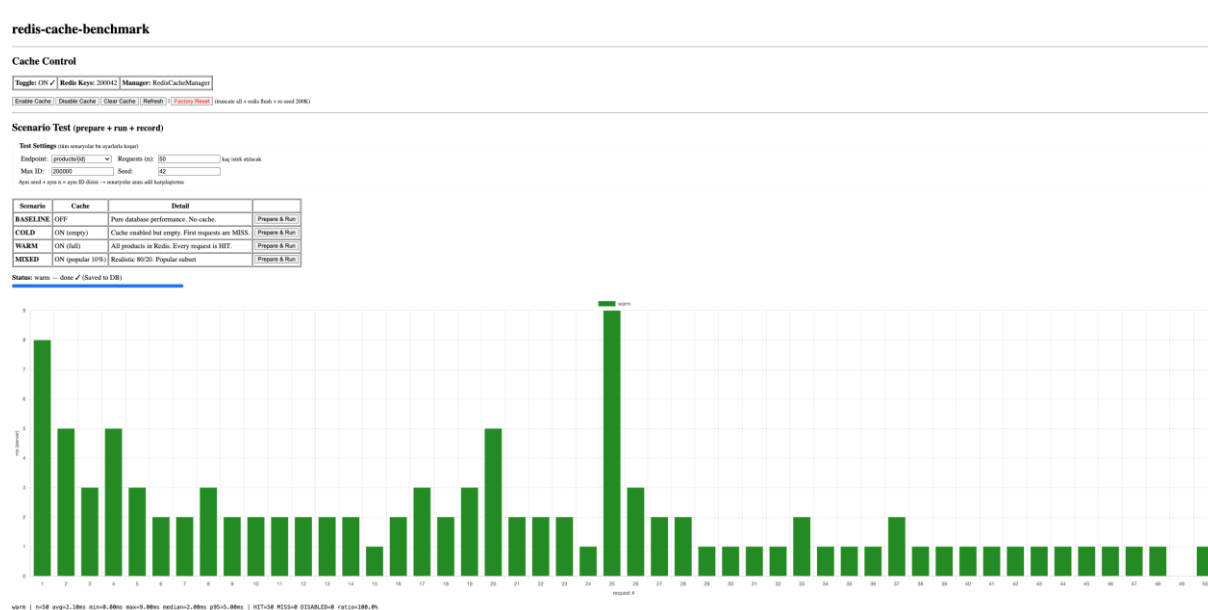


Figure 2. Web Interface — Scenario Test Panel

In addition to scenarios, we can also send requests to the system's endpoints that return a single response. Figure 3 shows a request sent for a single product. It also includes the hit or miss header information mentioned earlier.

id: 2

Run

→ 4.00 ms (server) [MISS]

```
{
  "id": 2,
  "name": "Rustic Plastic Watch",
  "description": "Possimus quo fuga praesentium hic exercitationem. Sed quos debitis.",
  "sku": "4755338260010",
  "price": 561.39,
  "stock": 692,
  "createdAt": "2025-07-30T19:15:13.17109",
  "category": {
    "id": 3,
    "name": "Tools 2",
    "parentId": null
  },
  "brand": {
    "id": 183,
    "name": "Prohaska and Sons 182",
    "country": "Andorra"
  }
}
```

GET /api/products?category=&page=&size=

category: 1

page: 0

size: 20

Run

→

GET /api/products/search?q=

q: co

Run

→

Request Log (last 5000)

Export CSV

Clear

#	Time	Path	Status	Server ms	Client ms	Cache
2	19:16:34	/api/products/2	200	4.00	7	MISS
1	19:16:28	/api/products/1	200	14.00	17	HIT

Figure 3. Web Interface — API Response and Cache Status

Although all these screens exist, a screen was needed where user could see the differences between the scenarios, which is why the screen shown in Figure 4 was created. This screen displays the results of previously conducted tests and includes information such as response times related to those tests.

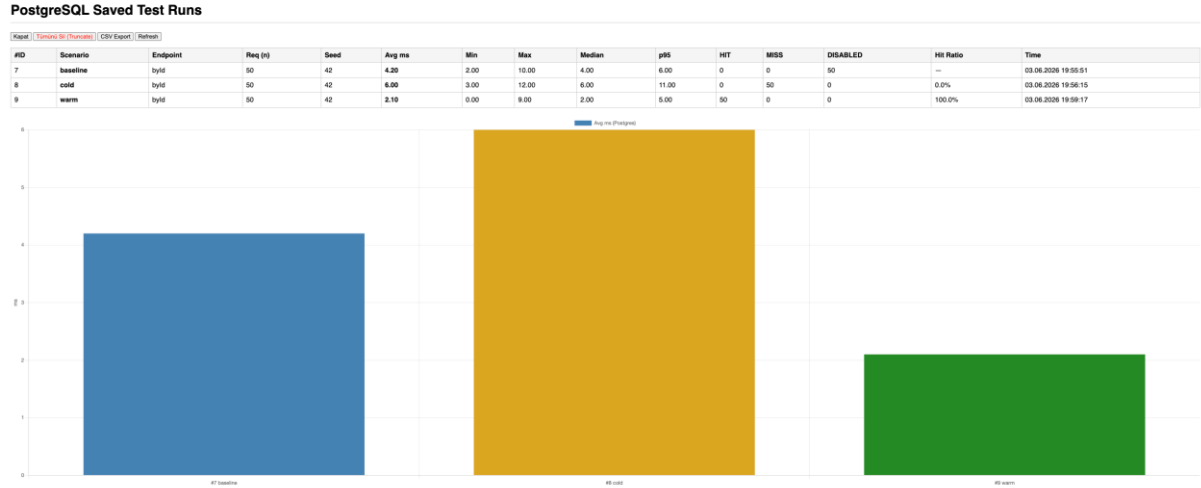


Figure 4. Web Interface — Test Results Dashboard

To ensure the accuracy of the tests, the PostgreSQL container is restarted via Docker Compose before each test scenario. The reason for this is to prevent the process from being manipulated—and thus producing a different result if data is returned in memory.

3.2. Test Environment

Redis 7 was used and installed via Docker. Additionally, Apache JMeter 5.6.3 was used to capture test results and run the tests. The following parameters were used for each test. 10,000 requests were sent per scenario

3.3. Dataset

The data was generated using the Java Faker library. A total of 200,000 records were generated, and this number can be adjusted via the application's configuration settings. Every time the application starts, it checks whether the required number of data points is present. If the number of data points is insufficient, it deletes all existing data and generates new data.

The database consists of three tables. The tables are related to each other. The Product table is related to both the Category and Brand tables. Table 2 shows the database structure.

Table 2. Database Structure

Table	Fields
product	id, name, description, sku, price, stock, created_at, category_id, brand_id
category	id, name, parent_id
brand	id, name, country

3.4. Test Scenarios

Three different scenarios were created. These scenarios are shown in Table 3.

Table 3. Test Scenarios

Scenario	Cache State	Redis State	Description
Baseline	Disabled	Empty	All requests go directly to PostgreSQL
Cold Cache	Enabled	Empty	Requests start as misses, cache fills gradually
Warm Cache	Enabled	Full	All requests served from Redis, no database contact

In the first scenario, Baseline scenario, requests go directly to the database. This scenario does not include any caching system. In other words, the simplest scenario.

In the second scenario, Cold Cache, Redis is enabled, meaning the caching system is active; however, since the requested data has not been written to the cache system yet, requests are first written to Redis, and then a response is returned.

In the third scenario, Warm Cache, Since all the requested data has already been written to the Redis cache system, this is expected to be the most efficient scenario. Requests are returned directly from the cache.

Figure 5 shows the process of each scenario.

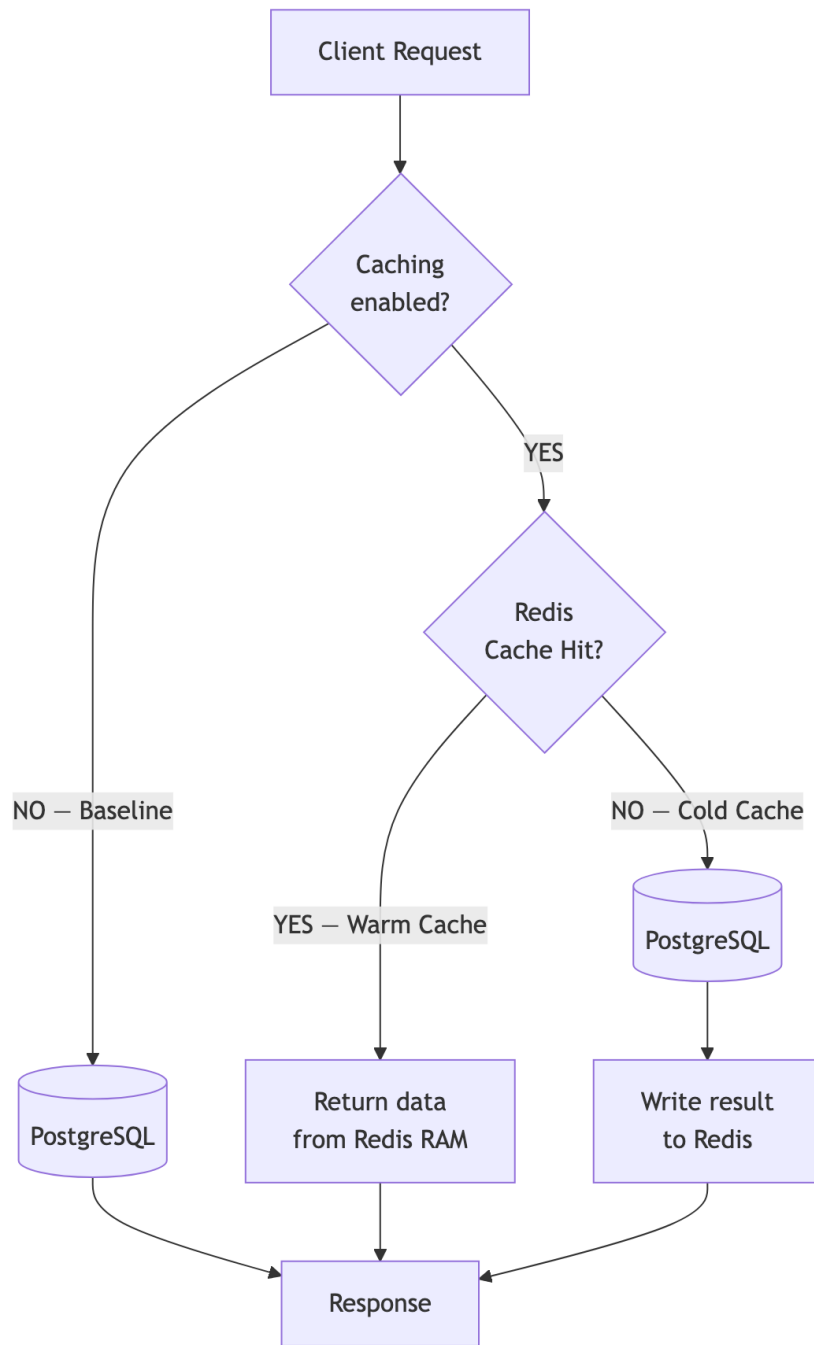


Figure 5. Request Flow Diagram for All Test Scenarios

3.5. Performance Metrics

The primary metric is response time. Response time was recorded in milliseconds. Milliseconds were used because the work was done in a local environment which has low latency. Response time in milisecond data was extracted from the CSV files provided by Apache JMeter after the tests.

4. FINDINGS

4.1. Test Results

In the Baseline scenario, the first of the scenarios, the average response time was measured at 1.21 ms. This was to be expected, as the response was returned from the database.

In the second scenario, “Cold Cache,” Redis was empty but active. In this scenario, the process of filling Redis with data affected the response time, which was measured at an average of 1.42 ms.

The final and third scenario is Warm Cache. Since the data was already stored in Redis, responses were returned direct from the cache. The average response time was measured at a remarkable 0.71.

Table 4 summarizes the test results.

Table 4. Test Results by Scenario

Scenario	Avg Response Time	Improvement vs Baseline
Baseline	1.21 ms	—
Cold Cache	1.42 ms	-17% (slower)
Warm Cache	0.71 ms	+41% (faster)

4.2. Analysis

Table 5 contains detailed results of these findings.

Table 5. Detailed Performance Metrics by Scenario

Metric	Baseline	Cold Cache	Warm Cache
Total Requests	10,000	10,000	10,000
Avg Response Time	1.21 ms	1.42 ms	0.71 ms
Max Response Time	51 ms	19 ms	14 ms
vs Baseline	—	−17% (slower)	+41% (faster)

Based on these results, in the cold cache scenario, even though the cache is active, the system's average response time of 1.42 ms due to the initial write-to-cache operation is worse than that of the baseline scenario, which reflects raw database performance. Additionally, as expected, the best result was achieved in the warm cache scenario with an average response time of 0.71 ms. This represents a 41% improvement compared to the baseline scenario. Finally, regarding maximum response times, while the cold cache scenario has the worst average response time, in the worst-case scenario, baseline has the worst result for the maximum response time. Also, warm cache scenario has the best result in maximum response time metric as expected. The reason the maximum response time is 19 ms in a cold cache scenario is that, as 10,000 requests are sent, the cache begins to fill up over time, and responses are returned from the cache.

5. CONCLUSION AND RECOMMENDATIONS

5.1. Summary of Findings

Redis directly affects how a Spring Boot application responds to requests and the response time of those requests. The way Redis is used directly impacts this response time. This study was designed to measure this effect in scenarios.

Redis stores data in RAM. Databases like PostgreSQL, which are used in the baseline scenario, read data from disk every time. This creates a fundamental difference between Redis and traditional databases. Therefore, this fundamental difference enables Redis to return responses more quickly.

Even before looking at the results, we can predict that Redis will have a positive impact. However, what matters here is measuring how the systems perform under different scenarios.

In the baseline scenario, the average response time was 1.21 ms and the maximum response time was 51 seconds. However, in the cold cache scenario, the process of filling the empty cache increased the average response time to 1.42 ms, resulting in a performance drop. This shows the importance of using the cache correctly. Additionally, in our final scenario, the warm cache where the application stores all data in the cache, the average response time dropped to 0.71 ms a significant improvement. However, in real-world scenarios, it is not applicable to keep all data in the cache. These tests were conducted to observe the direct impact of Redis.

5.2. Comparison with Literature

The results of our study are generally consistent with those of similar studies. For example, Zulfa et al. (2020) noted that performance was affected because the application had to go through a two-step process in the cache miss situation. This explains why our cold cache scenario produced poor results.

Pramudia et al. (2025) also noted that response times improved significantly when the data was already in the cache, which supports our findings in Warm cache scenario. Zbarcea et al. (2026) also noted that Redis cache systems lead to improved performance in Spring Boot applications. This is similar to the improvement observed in the test results conducted on Spring Boot Application used in this study. In addition, there are other solutions than caching for improving an application's performance, Gopal (2026) compared database optimization and caching and found that caching performs more effectively than database optimization, especially when response time is the primary factor, as in the application in this study.

5.3. Practical Implications

The impact of Redis is particularly visible when the same data is requested again and again. When the same request is made repeatedly, performance improves significantly because the data is stored in the Redis cache and the database is not in use at that moment. So, applications that experience changes in their data too many may find it more challenging to use Redis compared to others. In other words, in applications where write operations are performed more frequently than read operations, the Redis effect will be less.

5.4. Limitations

Since this study was conducted using a specific set of data that does not change frequently, the results do not fully represent the performance of a real-world application. Additionally, the tests were performed using a single query. Therefore, results for different queries with more complex workloads may affect the results. In real-world scenarios, factors such as network latency and website traffic exist, and these will also have an impact on latency. Since the data was generated using JavaFaker, it does not contain realistic information. However, the application and study are designed to provide a general idea.

Finally, in this study, tests were conducted using lazy loading. However, different caching systems are common. Privalov & Stupina (2024), mentioned in the study, use these systems. For example, in TTL-based caching systems, the cache data may expire and need to be refreshed.

5.5. Future Work

Research can be conducted on the following topics. Tests can be performed with additional endpoints and query types. Tests conducted with real data particularly using actual databases and data can perform more accurate results. Tests can also be conducted in an environment where Redis data changes frequently. Since the topic of cache expiration was not addressed in this study, this means the cache data that is no needed, expired. In such cases, the cached data present in the system can be included in the tests. Making the test environment more realistic could also be an interesting area of study. Such as, adding network latency etc. . For example, imagine a system containing products that users frequently interact with. Studies could be conducted to examine how the use of Redis in such scenarios would affect system performance.

REFERENCES

- Ayub, M. B., & Ali, N. (2018). Performance comparison of in-memory and disk-based databases using transaction processing performance council (TPC) benchmarking. *Journal of Internet and Information Systems*, 8(1), 1-8. <https://doi.org/10.5897/JIIS2018.0106>
- Bonteanu, A. M., & Tudose, C. (2024). Performance analysis and improvement for CRUD operations in relational databases from Java programs using JPA, Hibernate, Spring Data JPA. *Applied Sciences*, 14(7), 2743. <https://doi.org/10.3390/app14072743>
- Deshpande, R. A. (2025). Approaches to reducing REST API response time in high-traffic banking systems. *International Journal of Scientific Engineering and Science*, 9(8), 63-68.
- Gopal, A. (2026). Client and server caching strategies in modern mobile-first architectures: React Native, Next.js, and PostgreSQL. *SSRN*. <https://ssrn.com/abstract=6602728>
- Henao Villa, C. F., Echeverri Gutiérrez, C. A., Acosta Agudelo, L. C., García Arango, D. A., Garzon Cano, L. M., Santa Arboleda, E., & Hoyos Garcia, M. E. (2025). Optimizing microservices performance and scalability through automated monitoring with Kubernetes and Prometheus. *Ingénierie des Systèmes d'Information*, 30(2), 551-563. <https://doi.org/10.18280/isi.300225>
- Kanthed, S. (2023). Redis vs. Memcached in microservices architectures: Caching strategies. *International Journal of Multidisciplinary Research and Growth Evaluation*, 4(3), 1084-1091. <https://doi.org/10.54660/IJMRGE.2023.4.3.1084-1091>
- Mertz, J., & Nunes, I. (2020). Understanding application-level caching in web applications: A comprehensive introduction and survey of state-of-the-art. <https://doi.org/10.48550/arXiv.2011.00477>
- Pramudia, F. A., Zulfa, M. I., & Aliim, M. S. (2025). Evaluation of Redis in-memory based cache algorithm on web application data access performance using IRCache dataset. *Dinamika Rekayasa*, 21(2), 166-174. <https://doi.org/10.20884/1.dr.2025.21.2.166>
- Privalov, M. V., & Stupina, M. V. (2024). Improving web-oriented information systems efficiency using Redis caching mechanisms. *Indonesian Journal of Electrical Engineering and Computer Science*, 33(3), 1667-1675. <https://doi.org/10.11591/ijeecs.v33.i3.pp1667-1675>
- Yenugula, M., Kodam, R., & He, D. (2019). Performance and load testing: Tools and challenges. *International Journal of Engineering in Computer Science*, 1(1), 57-62. <https://doi.org/10.33545/26633582.2019.v1.i1a.102>
- Zbarcea, A., Tudose, C., & Boicea, A. (2026). Extending the migration from asynchronous to reactive programming in Java: A performance analysis of caching, CPU-bound, and blocking scenarios. *Applied Sciences*, 16, 90. <https://doi.org/10.3390/app16010090>

Zulfa, M. I., Fadli, A., & Wardhana, A. W. (2020). Application caching strategy based on in-memory using Redis server to accelerate relational data access. *Jurnal Teknologi dan Sistem Komputer*, 8(2), 157-163. <https://doi.org/10.14710/jtsiskom.8.2.2020.157-163>